

Enhancing Portfolio Optimization: Integrating Multivariate Distribution and Monte Carlo Simulation for Scenario Generation

Arun Govind, Aditya Santosh

Department of Quantitative Finance, Rutgers Business School - Newark

ABSTRACT

In a complex and dynamic market environment, managing and mitigating financial risks pose a significant challenge for financial institutions. The unpredictable nature of financial markets brings forward uncertainty, making it difficult for institutions to make decisions. Furthermore, achieving an optimal allocation of assets within portfolios, especially when faced with varying market scenarios is difficult and can impact the overall performance of investment portfolios. Our project proposes a solution that address these challenges by providing a C++ based optimization framework facilitating efficient allocation of assets within portfolios under various simulated scenarios, aiding financial institutions to achieve a balance between maximizing returns and minimizing risk. Thus, contributing to the overall effectiveness and resilience of financial institutions in managing their portfolios amid a dynamic economic landscape.

I. INTRODUCTION

“First, the only certainty is that there is no certainty. Second, every decision as a consequence is a matter of weighing probabilities. Third, despite uncertainty we must decide and we must act. And lastly, we need to judge decisions not only on the results, but how those decisions were made. – Robert E. Rubin”

The primary challenge in portfolio selection problems lies in demonstrating the efficacy of mathematical models as decision-making tools for reliable solutions in the face of uncertain future events. The effectiveness of these models is heavily reliant on the data input required and the methods employed to generate it. In portfolio selection problems, investors strive to choose a collection of securities that will optimize a performance measure while adhering to specific constraints. Numerous mathematical models for portfolio selection necessitate the availability of a range of scenarios, with each scenario representing a realization of a multivariate random variable that captures the rates of return for all the securities.

When making an investment decision, the primary factor that needs to be taken into account is the risk involved. This is because investment risk, which refers to the likelihood of incurring losses in relation to the expected return, cannot be eliminated in any specific investment. Consequently, the presence of risk not only impacts the rate of return on an investment but also influences an investor's decision when selecting investment tools. To effectively manage risk, portfolio selection is a recommended strategy as it allows for risk diversification, thereby minimizing potential losses. During the process of portfolio selection, careful consideration should be given to the types of stocks included and the availability of historical data on stock prices in order to make an optimal portfolio decision. Without this crucial information, the risk of experiencing losses in the investment decision may be heightened. Furthermore, the absence of a clear statement regarding the specific portfolio size being considered can pose a challenge when attempting to conduct an optimal portfolio selection. In cases where multiple portfolio models are available, it is essential to specify the criteria that need to be taken into account, such as correlation and covariance, in order to select the most suitable model.

Portfolio optimization is an essential aspect of investment management which aims to achieve the optimal balance between risk and return. The process involves selecting assets that form a portfolio while considering their potential returns, volatility, and correlation. It is tailored to the investor's preferences and investment goals, with the goal of maximizing returns while minimizing risk. Monte Carlo Simulation, a powerful computational technique used in various fields, including finance, addresses complex problems involving random variables and uncertainties. By running multiple simulations based on random sampling, statistical estimates and assessments can be derived. In the context of portfolio optimization, Monte Carlo Simulation provides valuable insights by generating a range of possible portfolio returns and risks based on specified parameters and assumptions.

The purpose of this comparative analysis is to evaluate different approaches for portfolio optimization, including the utilization of Monte Carlo Simulation. By comparing the outcomes of Monte Carlo Simulation with other optimization techniques, it enables us to gain a comprehensive understanding of the strengths and limitations of Monte Carlo Simulation in portfolio optimization. This analysis provides valuable insights that aid investors and portfolio managers in selecting the most suitable methodologies for optimizing their portfolios, taking into account the specific characteristics of their investment universe.

This paper aims to address the portfolio selection problem by providing a comprehensive quantitative analysis for effective portfolio management. The initial step involves calculating the rates of return, followed by the computation of the expected rates of return and the corresponding covariance matrix. Subsequently, the mean-variance optimization model, which is a quadratic programming model, is formulated and solved using quadratic programming techniques. In this model, the objective function seeks to minimize the risk associated with the portfolio selection. The decision variables in this model are the weights assigned to each asset in the portfolio, which are normalized based on the total investment amount. Additionally, a certain targeted return is assigned to the expected return. Overall, this study aims to offer valuable insights into managing portfolios by employing a rigorous quantitative approach. By considering the risk-return trade-off, the proposed model provides a framework for making

informed investment decisions and achieving optimal portfolio performance.

II. LITERATURE REVIEW - THEORETICAL FOUNDATIONS OF PORTFOLIO OPTIMIZATION

A. Probability Theory and Random Processes

Theoretical Foundations of Monte Carlo Simulation originate from the concepts of probability theory and random processes. Probability theory provides a mathematical framework to quantify the likelihood of different outcomes and events in a systematic manner. This theory equips us with the necessary tools to analyze the inherent uncertainty and randomness that exists within financial markets and asset returns ^[1]. On the other hand, random processes offer a mathematical representation of stochastic variables over a specific timeframe. By modeling these variables as random processes, we can simulate their behavior and generate various possible future scenarios ^[2].

Monte Carlo Simulation utilizes these underlying concepts to effectively model the uncertain elements of a system and generate random samples from probability distributions. These samples are then used to approximate the behavior of the system and derive statistical estimates of its properties. In the context of portfolio optimization, Monte Carlo Simulation facilitates the simulation of the random components of asset returns, such as market volatility and correlation, enabling an accurate assessment of their impact on portfolio performance ^[1].

B. Relevance in Financial Modeling

Performing financial simulations has gained immense importance in the field of financial modeling due to its ability to capture the inherent uncertainty and complexity of financial markets. Traditional analytical models are often based on simplistic assumptions and linear relationships that may not adequately reflect the dynamic nature of actual financial assets. In contrast, such simulations consider a wide range of possible inputs and account for random fluctuations in asset prices, volatility, and correlations.

By generating a large number of simulation scenarios in a portfolio strategy, the simulation provides an overview of the potential outcomes. This approach lets investors and portfolio managers to evaluate the likelihood and extent of various portfolio returns and risks, promoting informed

decision-making and effective risk management. Additionally, the simulations provide flexibility to incorporate constraints specific to the investment domain, enabling a customized approach to portfolio optimization.

C. Markowitz's Mean Variance Optimization Process

In recent decades, there has been a significant surge in the popularity of quantitative approaches to asset management. Among the various algorithms and strategies developed, portfolio selection models have garnered considerable attention. The fundamental objective of portfolio investing is to determine the optimal allocation of weights to a given set of assets, with the aim of maximizing future portfolio returns while simultaneously minimizing investment risk. The exploration of this field commences with Markowitz's Mean-Variance optimization process.

The concept of mean-variance-based portfolio selection remains a fundamental principle in contemporary asset management. It is founded on the belief that rational investors make decisions regarding risky assets solely based on expected return and risk, with risk being measured as portfolio variance. The theoretical underpinning of this framework is robust under two conditions: either investors exhibit quadratic utility, disregarding non-normality in the data, or all higher moments of the portfolio distribution can be expressed as a function of mean and variance, thereby satisfying the mean-variance criterion for all optimal solutions. However, it is important to note that the "optimality" of mean-variance portfolios assumes investors exist in a one-period world, whereas in reality, their investment horizon extends beyond a single period. In actuality, markets are subject to constant fluctuations over time, and investors must adapt their allocation and take corrective actions in response to inflows and outflows.

Considering a portfolio of n assets with weights $W = (w_1, \dots, w_n)$, returns $R = (r_1, \dots, r_n)$ and portfolio returns:

$$r_p = W R^T$$

the standard Mean-Variance optimization problem consists in minimizing the portfolios' variance σ_p for fixed levels of expected returns:

$$E[r_p] = \bar{r}_p:$$

$$\begin{aligned} \min_W \quad & \sigma_p^2 = W \Sigma W^T \\ \text{s.t.} \quad & E[r_p] = \bar{r}_p, \\ \text{and} \quad & W \mathbf{1} = 1, \end{aligned}$$

where $\Sigma \in \mathbb{R}^{n \times n}$ is the assets' covariance matrix and $\mathbf{1} \in \mathbb{R}^{n \times 1}$ is a basis column vector with all elements equal 1. Solving for W for different values of r_p , one can obtain the optimal weights corresponding to different portfolio expected returns r_p yielding the so-called efficient frontier.

The estimation error is quantified by measuring how well the functional form of the multivariate probability distribution, $f_\theta(\mathbf{x})$ defined via the parameters θ estimated in-sample, describes the actual data out-of-sample. The statistical measure that describes how likely are observations to belong to the estimated probability function is the likelihood. The likelihood principle is a cornerstone of statistical analysis in that maximum likelihood estimators are guaranteed to be asymptotically efficient under mild conditions.

The logarithm of the likelihood for the normal case is proportional to:

$$\ln \tau(\theta; X_t) = \ln |J| - (X_t - \mu)^T J (X_t - \mu) + k$$

where $X_t = (x_{t,1}, x_{t,2}, \dots, x_{t,n})$ is the n -dimensional multivariate returns observation vector at time t ; θ is the model parameter set, which includes μ the vector of means and J the generalized precision matrix and; k is a constant which is independent from μ , J or X_t .

In the multivariate normal case $J = \Sigma^{-1}$ is the inverse of the covariance. Our results generalize to other elliptical distributions with defined covariance where J is proportional to the inverse covariance, which we assume is defined and invertible.

III. METHODOLOGY- IMPLEMENTATION IN C++

A. Overview of Algorithmic Steps

The implementation involves several key steps beginning with user, they define the investment universe by selecting a set of assets to consider for portfolio construction. These assets may include stocks, bonds, commodities or other

financial instruments given that historical prices are available for the afore mentioned assets. Next, user specify desired portfolio characteristics such as expected returns, risk tolerance and any specific limits. Once the input parameters are defined, a simulation algorithm is executed to generate a series of scenarios based on random samples from a specific probability distribution and a suitable portfolio with calculated weights is given that adheres to these characteristics.

During each iteration of the simulation, portfolio weights are randomly generated, subject to prescribed constraints. These weights determine the allocation of assets within the portfolio. The expected return, volatility, and correlation of the selected assets are then used to calculate the return and risk of the portfolio. This process is repeated for a large number of iterations, often thousands or millions of times, to generate a wide range of portfolio results. Finally, statistical metrics such as mean, standard deviation, and skewness are calculated based on simulated returns to quantify key characteristics of the portfolio.

i) Dataset

In our project, we have meticulously gathered weekly stock data spanning for a year from 2021-2022 for a diverse portfolio of 10 companies. The selection includes a mix of large, mid, and small-cap enterprises.

1. Large Caps:

- Apple (AAPL)
- Exxon Mobil (XOM)
- Microsoft (MSFT)
- Walmart (WMT)
- Berkshire Hathaway (BRK.B)

2. Mid-Caps:

- U-Haul (UHAL)
- Etsy (ETSY)
- XPO (XPO)

3. Small Caps:

- Goldman Sachs BDC, Inc (GS)
- Footlocker (FL)

The data extraction process was conducted using Yahoo Finance, ensuring a comprehensive dataset for each stock. **An essential decision point in our analysis is whether to consider the past 1 year of data or the entire available dataset.** Currently we are using just a year from the available datasets.

For each stock returns, we calculate key statistical measures:

- Mean
- Standard Deviation
- Skewness
- Kurtosis

These metrics provide a robust understanding of the distribution characteristics of each stock's returns. Given our focus on portfolio risk optimization, we concentrate on directly predicting return covariances from the calculated returns respectively.

ii) Computer Specifications

For this project, we have been working with C++ (14) and the GCC compiler. Specifications of the hardware include an 16GB RAM, an Intel i7 (12th generation), NVIDIA GEFORCE 3060RTX and a 1TB solid-state drive.

iii) Normal Distribution Consideration

While acknowledging the theoretical normal distribution of returns ($Z(0,1)$) as the sample size increases, we opt for a practical approach. The mean and standard deviation derived from historical data serve as the parameters for plotting the normal distribution curve for each stock. This decision aligns with the specifics of our dataset and ensures a more accurate representation of the observed patterns.

Next, we sample scenarios for each stock from the respective normal distribution plots. This approach allows us to simulate various market conditions and assess the potential outcomes for each company.

B. Coding Best Practices for Simulation Efficiency

To achieve optimal computational efficiency when implementing Monte Carlo Simulation in C++, several coding best practices should be followed:

1. **Vectorization:** We utilized the vectorized operations available in C++ to perform computations on data structures as a whole which significantly sped up the simulation process and enhanced our overall computational efficiency.
2. **Memory Management:** We efficiently managed memory allocation and deallocation to minimize unnecessary overhead. Avoiding unnecessary memory

copy operations to maintain simulation speed and reducing memory usage was thoroughly taken care of.

3. **Random Number Generation:** Used high-quality random number generators to ensure the randomness and statistical properties of the generated scenarios. The C++ standard library provided various random number generation functions which were used in Monte Carlo Simulation implementations.
4. **Code Optimization:** Utilized code optimization techniques, such as compiler optimizations and profiling tools, to identify and removed bottlenecks in the implementation.

By following these coding best practices, the implementation of Monte Carlo Simulation in C++ can achieve high-performance computing capabilities, allowing for efficient and accurate portfolio analysis in real-world financial scenarios.

IV. RESULTS & INFERENCE

The following is the output that we obtain after reading in some data using C++ and applying the relevant code.

TABLE 1: AVERAGE CORRELATION OF SELECTED STOCKS

Stock	Mean	Standard Deviation	Skewness	Kurtosis
Exxon	0.0095	0.04273	0.1793	3.85099
Microsoft	0.0015	0.03542	0.04849	3.89187
Hathway	0.0030	0.0275	-0.2467	4.80997
Apple	0.0005	0.03601	0.05374	2.92213
Walmart	0.0002	0.03203	-1.981	13.9017
Etsy	0.0006	0.09055	0.3011	3.39544
UHaul	0.0032	0.04009	0.4079	3.5136
XPO	-0.001	0.05551	0.4313	4.34878
GoldmanS	-0.002	0.0255123	-0.8809	5.12673
Footlocker	0.0006	0.0635528	-0.3939	4.90191

Standard Deviation: The standard deviation of a stock measures its volatility, indicating how much its returns fluctuate from the expected return. GoldmanS has a low standard deviation of 0.0255, signaling relatively stable returns, while Etsy's higher standard deviation of 0.0905 suggests more variability and potential volatility.

Skewness: Skewness in stock returns indicates the distribution's asymmetry. XPO demonstrates a notably positive skewness of 0.431, signifying a distribution tilted to the right, indicating a higher probability of positive returns. Conversely, Walmart exhibits a substantial negative skewness of -1.981, suggesting a left-tilted distribution with a higher likelihood of negative returns.

Kurtosis: Microsoft displays a kurtosis of 3.89, indicating leptokurtic behavior similar to Exxon, with fatter tails and a more peaked center compared to a normal distribution. On the other hand, Walmart exhibits an exceptionally high kurtosis of 13.901, signaling a very leptokurtic distribution, implying a significant probability of extreme positive and negative returns.

With the collected distributions, our goal is to build a range of 100,000 to 1,000,000 scenarios (depending on processing power) to estimate the distribution of returns for the portfolio. This will allow us to compute the portfolio's characteristics. Below is an illustration of a solitary situation developed by our team.

Scenarios are created for a period of one year, specifically 52 weeks. This results in a matrix of returns with dimensions of 10 rows and 52 columns. The mean returns of the 10 stocks are shown below:

These are the mean returns of the stocks:

```

0.0088855924470961
0.00742886935978583
0.00594285532874096
0.00568061941351132
-0.000143344989835251
0.00602086673542268
0.00868692115764688
0.000575527147660427
0.000576255559108201
0.000798979217909919

```

Returns are calculated by taking the current stock price and subtracting the previous stock price, dividing it by the previous stock price.

The covariance matrix of the stocks:

```
This is the covariance matrix of the stocks:
0.00856903148236738 -0.00204556160222922 0.00207952052926704 -0.00192401901820593 -0.000209
0.000930528756618731 0.0034452956122069 0.0034452956122069 -0.00204556160222922 0.0034452956122069
-0.00204556160222922 0.0034452956122069 -0.00204556160222922 0.0034452956122069 -0.00204556160222922
0.000308316152377853 0.00194433931520789 0.00194433931520789 -0.00204556160222922 0.0034452956122069
0.00207952052926704 -8.11994359802311e-05 0.00238931514305584 -1.88434106952534e-05 0.00148
9.44476305258809e-05 0.00195238731799118 0.00195238731799118 -1.88434106952534e-05 0.00148
-0.00192401901820593 0.00261471564837293 -1.88434106952534e-05 0.00438483399029437 0.00101
0.00011840408566465 0.000342136027067533 0.000342136027067533 0.00101840408566465 0.00011840408566465
-0.00020952052926704 0.00106119533281841 0.00106119533281841 0.00438483399029437 0.00101
0.000135691824304686 0.00117023342527252 0.00117023342527252 0.00101840408566465 0.00011840408566465
0.00219788993350255 0.0050694886055366 2.86524823648483e-05 0.0036581413645731 0.00195
6.26163567761434e-05 0.0097339965512218 2.86524823648483e-05 0.0036581413645731 0.00195
0.0022977265102872 7.50484709499291e-05 0.00134720371440639 -0.000751941464592889 0.000130
0.000687565772264056 0.00354547347436332 0.00134720371440639 -0.000751941464592889 0.000130
0.00108349213519785 0.00198997183045736 0.00147543840883986 0.00249783231651351 0.00112
0.00108257192033452 0.00408124822854154 0.00147543840883986 0.00249783231651351 0.00112
0.000930528756618731 -0.000308316152377853 9.44476305258809e-05 0.00011840408566465 -0.000135
0.00197973627910122 0.000534763971959165 0.00197973627910122 0.000534763971959165 0.00197973627910122
0.0034452956122069 0.00194433931520789 0.0034452956122069 0.00194433931520789 0.0034452956122069
0.000534763971959165 0.00117023342527252 0.000534763971959165 0.00117023342527252 0.000534763971959165
```

We have calculated the inverse of the covariance of the matrix using the eigen library. Eigen library uses Full Pivot LU decomposition of the matrix to compute the same. We find the upper triangle (U), lower triangle (L) and permutation matrices (P and Q).

After calculating the inverse covariance matrix to check if we have right, $A \cdot A^{-1} = I$

Then, we get the Identity matrix below:

```
Should be 10x10 I matrix
1 -3.05311e-16 1.11022e-16 1.05818e-16 5.55112e-17 2.49777e-17 1.11022e-16 1.05818e-16 5.55112e-17 2.49777e-17
-2.39392e-16 1 -1.249e-16 -1.249e-16 1.15359e-16 3.26128e-16 1.15359e-16 1.15359e-16 3.26128e-16 1.15359e-16
1.49186e-16 -6.93889e-17 1 8.32667e-17 -8.76035e-17 1.38778e-17 1.38778e-17 1.38778e-17 1.38778e-17 1.38778e-17
-5.63785e-17 -3.57353e-16 -2.06432e-16 1 4.98733e-18 -3.81639e-17 9.44476e-18 -3.81639e-17 9.44476e-18 -3.81639e-17
-6.93889e-18 -9.02056e-17 -1.66533e-16 4.16334e-17 1 -3.1225e-17 1.38778e-17 1.38778e-17 1.38778e-17 1.38778e-17
1.11022e-16 -4.44089e-16 2.22045e-16 2.77556e-16 6.93889e-18 1 5.55112e-17 5.55112e-17 5.55112e-17 5.55112e-17
4.85723e-17 -6.93889e-17 -1.38778e-16 2.498e-16 -2.04697e-16 0 1 1.11022e-16 1.11022e-16 1.11022e-16
2.77556e-17 -3.05311e-16 -8.32667e-17 1.94289e-16 2.08167e-17 4.16334e-17 1.11022e-16 1.11022e-16 1.11022e-16 1.11022e-16
-1.04083e-17 -2.77556e-17 -1.56125e-16 6.93889e-17 1.69136e-17 -5.20417e-18 1.11022e-16 1.11022e-16 1.11022e-16 1.11022e-16
-8.32667e-17 -1.11022e-16 3.33067e-16 2.22045e-16 -1.59595e-16 1.11022e-16 1.11022e-16 1.11022e-16 1.11022e-16 1.11022e-16
```

The user has to enter a target mean; the program will provide the optimum weights required to achieve the target.

Enter Target Mean: 0.0

We receive:

```
Enter Target Mean:0.02
These are the optimal weights for specified target mean:
0.180206289149746
0.399471310310621
0.192793137400713
0.222417980345584
-0.0859410625231615
-0.00162483779976866
0.22659653784438
-0.000693656775359885
-0.053558041305895
-0.079667656646859

Sum of weights(Should be 1):
1
```

```
// Function to calculate optimal weights of a stock
MatrixXd& Calculate_optimal_weights(const MatrixXd& Covariance_mat, const MatrixXd& meanMatrix) {
    Rand::MvNormalGen(double, 10) gen1( meanMatrix, Covariance_mat );
    double TARGET_MEAN;

    cout << "Enter Target Mean:";
    cin >> TARGET_MEAN;

    // We have set stack limit to 262144000 (250Mb) to avoid overflow
    MatrixXd OPT_WEIGHTS;
    for (int i = 0; i < 10; i++) {
        MatrixXd PortfolioWeights;
        size_t seed = rand();
        Rand::P8_mt19937_64 urng( seed );
        MatrixXd double, 10, -1> Samples = gen1.generate(urng, 52);
        //cout << "These are the scenarios:\n" << endl;
        //cout << Samples << endl;
    }
}
```

The above code, we have Eigen Rand library to generate a 10-dimensional multi variate normal distribution. The scenarios are sampled from this distribution. We have used **srand** to generate a random seed and put that into a random generator **P8_mt19937** to get different scenarios every time.

Monte-Carlo simulation:

Each scenario indicates a projection of 52 weekly returns of each individual stock, giving a 10x52 matrix.

Case 1 (Simulation run for 10 scenarios):

```
1 // we have set stack limit to 262144000 (250Mb) to avoid overflow
2 MatrixXd double, 10, 10> OPT_WEIGHTS;
3 for (int i = 0; i < 10; i++) {
4     MatrixXd PortfolioWeights;
5     size_t seed = rand();
6     Rand::P8_mt19937_64 urng( seed );
7     MatrixXd double, 10, -1> Samples = gen1.generate(urng, 52);
8     //cout << "These are the scenarios:\n" << endl;
9     //cout << Samples << endl;
10 }
11 // calculate covariance matrix of each scenario
12 MatrixXd covariance_Samples = Calculate_MeanMatrix(Samples);
13 MatrixXd inv_covariance_Samples = Calculate_Inverse_MeanMatrix(covariance_Samples);
14 //cout << inv_covariance_Samples << endl;
15 //cout << covariance_Samples << endl;
16 //cout << "Sum of weights(Should be 1):" << endl;
17 //cout << OPT_WEIGHTS << endl;
18 //cout << "Sum of weights(Should be 1):" << endl;
19 //cout << "Sum of weights(Should be 1):" << endl;
20 //cout << "Sum of weights(Should be 1):" << endl;
21 //cout << "Sum of weights(Should be 1):" << endl;
22 //cout << "Sum of weights(Should be 1):" << endl;
23 //cout << "Sum of weights(Should be 1):" << endl;
24 //cout << "Sum of weights(Should be 1):" << endl;
25 //cout << "Sum of weights(Should be 1):" << endl;
26 //cout << "Sum of weights(Should be 1):" << endl;
27 //cout << "Sum of weights(Should be 1):" << endl;
28 //cout << "Sum of weights(Should be 1):" << endl;
29 //cout << "Sum of weights(Should be 1):" << endl;
30 //cout << "Sum of weights(Should be 1):" << endl;
31 //cout << "Sum of weights(Should be 1):" << endl;
32 //cout << "Sum of weights(Should be 1):" << endl;
33 //cout << "Sum of weights(Should be 1):" << endl;
34 //cout << "Sum of weights(Should be 1):" << endl;
35 //cout << "Sum of weights(Should be 1):" << endl;
36 //cout << "Sum of weights(Should be 1):" << endl;
37 //cout << "Sum of weights(Should be 1):" << endl;
38 //cout << "Sum of weights(Should be 1):" << endl;
39 //cout << "Sum of weights(Should be 1):" << endl;
40 //cout << "Sum of weights(Should be 1):" << endl;
41 //cout << "Sum of weights(Should be 1):" << endl;
42 //cout << "Sum of weights(Should be 1):" << endl;
43 //cout << "Sum of weights(Should be 1):" << endl;
44 //cout << "Sum of weights(Should be 1):" << endl;
45 //cout << "Sum of weights(Should be 1):" << endl;
46 //cout << "Sum of weights(Should be 1):" << endl;
47 //cout << "Sum of weights(Should be 1):" << endl;
48 //cout << "Sum of weights(Should be 1):" << endl;
49 //cout << "Sum of weights(Should be 1):" << endl;
50 //cout << "Sum of weights(Should be 1):" << endl;
51 //cout << "Sum of weights(Should be 1):" << endl;
52 //cout << "Sum of weights(Should be 1):" << endl;
53 //cout << "Sum of weights(Should be 1):" << endl;
54 //cout << "Sum of weights(Should be 1):" << endl;
55 //cout << "Sum of weights(Should be 1):" << endl;
56 //cout << "Sum of weights(Should be 1):" << endl;
57 //cout << "Sum of weights(Should be 1):" << endl;
58 //cout << "Sum of weights(Should be 1):" << endl;
59 //cout << "Sum of weights(Should be 1):" << endl;
60 //cout << "Sum of weights(Should be 1):" << endl;
61 //cout << "Sum of weights(Should be 1):" << endl;
62 //cout << "Sum of weights(Should be 1):" << endl;
63 //cout << "Sum of weights(Should be 1):" << endl;
64 //cout << "Sum of weights(Should be 1):" << endl;
65 //cout << "Sum of weights(Should be 1):" << endl;
66 //cout << "Sum of weights(Should be 1):" << endl;
67 //cout << "Sum of weights(Should be 1):" << endl;
68 //cout << "Sum of weights(Should be 1):" << endl;
69 //cout << "Sum of weights(Should be 1):" << endl;
70 //cout << "Sum of weights(Should be 1):" << endl;
71 //cout << "Sum of weights(Should be 1):" << endl;
72 //cout << "Sum of weights(Should be 1):" << endl;
73 //cout << "Sum of weights(Should be 1):" << endl;
74 //cout << "Sum of weights(Should be 1):" << endl;
75 //cout << "Sum of weights(Should be 1):" << endl;
76 //cout << "Sum of weights(Should be 1):" << endl;
77 //cout << "Sum of weights(Should be 1):" << endl;
78 //cout << "Sum of weights(Should be 1):" << endl;
79 //cout << "Sum of weights(Should be 1):" << endl;
80 //cout << "Sum of weights(Should be 1):" << endl;
81 //cout << "Sum of weights(Should be 1):" << endl;
82 //cout << "Sum of weights(Should be 1):" << endl;
83 //cout << "Sum of weights(Should be 1):" << endl;
84 //cout << "Sum of weights(Should be 1):" << endl;
85 //cout << "Sum of weights(Should be 1):" << endl;
86 //cout << "Sum of weights(Should be 1):" << endl;
87 //cout << "Sum of weights(Should be 1):" << endl;
88 //cout << "Sum of weights(Should be 1):" << endl;
89 //cout << "Sum of weights(Should be 1):" << endl;
90 //cout << "Sum of weights(Should be 1):" << endl;
91 //cout << "Sum of weights(Should be 1):" << endl;
92 //cout << "Sum of weights(Should be 1):" << endl;
93 //cout << "Sum of weights(Should be 1):" << endl;
94 //cout << "Sum of weights(Should be 1):" << endl;
95 //cout << "Sum of weights(Should be 1):" << endl;
96 //cout << "Sum of weights(Should be 1):" << endl;
97 //cout << "Sum of weights(Should be 1):" << endl;
98 //cout << "Sum of weights(Should be 1):" << endl;
99 //cout << "Sum of weights(Should be 1):" << endl;
100 //cout << "Sum of weights(Should be 1):" << endl;
101 //cout << "Sum of weights(Should be 1):" << endl;
102 //cout << "Sum of weights(Should be 1):" << endl;
103 //cout << "Sum of weights(Should be 1):" << endl;
104 //cout << "Sum of weights(Should be 1):" << endl;
105 //cout << "Sum of weights(Should be 1):" << endl;
106 //cout << "Sum of weights(Should be 1):" << endl;
107 //cout << "Sum of weights(Should be 1):" << endl;
108 //cout << "Sum of weights(Should be 1):" << endl;
109 //cout << "Sum of weights(Should be 1):" << endl;
110 //cout << "Sum of weights(Should be 1):" << endl;
111 //cout << "Sum of weights(Should be 1):" << endl;
112 //cout << "Sum of weights(Should be 1):" << endl;
113 //cout << "Sum of weights(Should be 1):" << endl;
114 //cout << "Sum of weights(Should be 1):" << endl;
115 //cout << "Sum of weights(Should be 1):" << endl;
116 //cout << "Sum of weights(Should be 1):" << endl;
117 //cout << "Sum of weights(Should be 1):" << endl;
118 //cout << "Sum of weights(Should be 1):" << endl;
119 //cout << "Sum of weights(Should be 1):" << endl;
120 //cout << "Sum of weights(Should be 1):" << endl;
121 //cout << "Sum of weights(Should be 1):" << endl;
122 //cout << "Sum of weights(Should be 1):" << endl;
123 //cout << "Sum of weights(Should be 1):" << endl;
124 //cout << "Sum of weights(Should be 1):" << endl;
125 //cout << "Sum of weights(Should be 1):" << endl;
126 //cout << "Sum of weights(Should be 1):" << endl;
127 //cout << "Sum of weights(Should be 1):" << endl;
128 //cout << "Sum of weights(Should be 1):" << endl;
129 //cout << "Sum of weights(Should be 1):" << endl;
130 //cout << "Sum of weights(Should be 1):" << endl;
131 //cout << "Sum of weights(Should be 1):" << endl;
132 //cout << "Sum of weights(Should be 1):" << endl;
133 //cout << "Sum of weights(Should be 1):" << endl;
134 //cout << "Sum of weights(Should be 1):" << endl;
135 //cout << "Sum of weights(Should be 1):" << endl;
136 //cout << "Sum of weights(Should be 1):" << endl;
137 //cout << "Sum of weights(Should be 1):" << endl;
138 //cout << "Sum of weights(Should be 1):" << endl;
139 //cout << "Sum of weights(Should be 1):" << endl;
140 //cout << "Sum of weights(Should be 1):" << endl;
141 //cout << "Sum of weights(Should be 1):" << endl;
142 //cout << "Sum of weights(Should be 1):" << endl;
143 //cout << "Sum of weights(Should be 1):" << endl;
144 //cout << "Sum of weights(Should be 1):" << endl;
145 //cout << "Sum of weights(Should be 1):" << endl;
146 //cout << "Sum of weights(Should be 1):" << endl;
147 //cout << "Sum of weights(Should be 1):" << endl;
148 //cout << "Sum of weights(Should be 1):" << endl;
149 //cout << "Sum of weights(Should be 1):" << endl;
150 //cout << "Sum of weights(Should be 1):" << endl;
151 //cout << "Sum of weights(Should be 1):" << endl;
152 //cout << "Sum of weights(Should be 1):" << endl;
153 //cout << "Sum of weights(Should be 1):" << endl;
154 //cout << "Sum of weights(Should be 1):" << endl;
155 //cout << "Sum of weights(Should be 1):" << endl;
156 //cout << "Sum of weights(Should be 1):" << endl;
157 //cout << "Sum of weights(Should be 1):" << endl;
158 //cout << "Sum of weights(Should be 1):" << endl;
159 //cout << "Sum of weights(Should be 1):" << endl;
160 //cout << "Sum of weights(Should be 1):" << endl;
161 //cout << "Sum of weights(Should be 1):" << endl;
162 //cout << "Sum of weights(Should be 1):" << endl;
163 //cout << "Sum of weights(Should be 1):" << endl;
164 //cout << "Sum of weights(Should be 1):" << endl;
165 //cout << "Sum of weights(Should be 1):" << endl;
166 //cout << "Sum of weights(Should be 1):" << endl;
167 //cout << "Sum of weights(Should be 1):" << endl;
168 //cout << "Sum of weights(Should be 1):" << endl;
169 //cout << "Sum of weights(Should be 1):" << endl;
170 //cout << "Sum of weights(Should be 1):" << endl;
171 //cout << "Sum of weights(Should be 1):" << endl;
172 //cout << "Sum of weights(Should be 1):" << endl;
173 //cout << "Sum of weights(Should be 1):" << endl;
174 //cout << "Sum of weights(Should be 1):" << endl;
175 //cout << "Sum of weights(Should be 1):" << endl;
176 //cout << "Sum of weights(Should be 1):" << endl;
177 //cout << "Sum of weights(Should be 1):" << endl;
178 //cout << "Sum of weights(Should be 1):" << endl;
179 //cout << "Sum of weights(Should be 1):" << endl;
180 //cout << "Sum of weights(Should be 1):" << endl;
181 //cout << "Sum of weights(Should be 1):" << endl;
182 //cout << "Sum of weights(Should be 1):" << endl;
183 //cout << "Sum of weights(Should be 1):" << endl;
184 //cout << "Sum of weights(Should be 1):" << endl;
185 //cout << "Sum of weights(Should be 1):" << endl;
186 //cout << "Sum of weights(Should be 1):" << endl;
187 //cout << "Sum of weights(Should be 1):" << endl;
188 //cout << "Sum of weights(Should be 1):" << endl;
189 //cout << "Sum of weights(Should be 1):" << endl;
190 //cout << "Sum of weights(Should be 1):" << endl;
191 //cout << "Sum of weights(Should be 1):" << endl;
192 //cout << "Sum of weights(Should be 1):" << endl;
193 //cout << "Sum of weights(Should be 1):" << endl;
194 //cout << "Sum of weights(Should be 1):" << endl;
195 //cout << "Sum of weights(Should be 1):" << endl;
196 //cout << "Sum of weights(Should be 1):" << endl;
197 //cout << "Sum of weights(Should be 1):" << endl;
198 //cout << "Sum of weights(Should be 1):" << endl;
199 //cout << "Sum of weights(Should be 1):" << endl;
200 //cout << "Sum of weights(Should be 1):" << endl;
201 //cout << "Sum of weights(Should be 1):" << endl;
202 //cout << "Sum of weights(Should be 1):" << endl;
203 //cout << "Sum of weights(Should be 1):" << endl;
204 //cout << "Sum of weights(Should be 1):" << endl;
205 //cout << "Sum of weights(Should be 1):" << endl;
206 //cout << "Sum of weights(Should be 1):" << endl;
207 //cout << "Sum of weights(Should be 1):" << endl;
208 //cout << "Sum of weights(Should be 1):" << endl;
209 //cout << "Sum of weights(Should be 1):" << endl;
210 //cout << "Sum of weights(Should be 1):" << endl;
211 //cout << "Sum of weights(Should be 1):" << endl;
212 //cout << "Sum of weights(Should be 1):" << endl;
213 //cout << "Sum of weights(Should be 1):" << endl;
214 //cout << "Sum of weights(Should be 1):" << endl;
215 //cout << "Sum of weights(Should be 1):" << endl;
216 //cout << "Sum of weights(Should be 1):" << endl;
217 //cout << "Sum of weights(Should be 1):" << endl;
218 //cout << "Sum of weights(Should be 1):" << endl;
219 //cout << "Sum of weights(Should be 1):" << endl;
220 //cout << "Sum of weights(Should be 1):" << endl;
221 //cout << "Sum of weights(Should be 1):" << endl;
222 //cout << "Sum of weights(Should be 1):" << endl;
223 //cout << "Sum of weights(Should be 1):" << endl;
224 //cout << "Sum of weights(Should be 1):" << endl;
225 //cout << "Sum of weights(Should be 1):" << endl;
226 //cout << "Sum of weights(Should be 1):" << endl;
227 //cout << "Sum of weights(Should be 1):" << endl;
228 //cout << "Sum of weights(Should be 1):" << endl;
229 //cout << "Sum of weights(Should be 1):" << endl;
230 //cout << "Sum of weights(Should be 1):" << endl;
231 //cout << "Sum of weights(Should be 1):" << endl;
232 //cout << "Sum of weights(Should be 1):" << endl;
233 //cout << "Sum of weights(Should be 1):" << endl;
234 //cout << "Sum of weights(Should be 1):" << endl;
235 //cout << "Sum of weights(Should be 1):" << endl;
236 //cout << "Sum of weights(Should be 1):" << endl;
237 //cout << "Sum of weights(Should be 1):" << endl;
238 //cout << "Sum of weights(Should be 1):" << endl;
239 //cout << "Sum of weights(Should be 1):" << endl;
240 //cout << "Sum of weights(Should be 1):" << endl;
241 //cout << "Sum of weights(Should be 1):" << endl;
242 //cout << "Sum of weights(Should be 1):" << endl;
243 //cout << "Sum of weights(Should be 1):" << endl;
244 //cout << "Sum of weights(Should be 1):" << endl;
245 //cout << "Sum of weights(Should be 1):" << endl;
246 //cout << "Sum of weights(Should be 1):" << endl;
247 //cout << "Sum of weights(Should be 1):" << endl;
248 //cout << "Sum of weights(Should be 1):" << endl;
249 //cout << "Sum of weights(Should be 1):" << endl;
250 //cout << "Sum of weights(Should be 1):" << endl;
251 //cout << "Sum of weights(Should be 1):" << endl;
252 //cout << "Sum of weights(Should be 1):" << endl;
253 //cout << "Sum of weights(Should be 1):" << endl;
254 //cout << "Sum of weights(Should be 1):" << endl;
255 //cout << "Sum of weights(Should be 1):" << endl;
256 //cout << "Sum of weights(Should be 1):" << endl;
257 //cout << "Sum of weights(Should be 1):" << endl;
258 //cout << "Sum of weights(Should be 1):" << endl;
259 //cout << "Sum of weights(Should be 1):" << endl;
260 //cout << "Sum of weights(Should be 1):" << endl;
261 //cout << "Sum of weights(Should be 1):" << endl;
262 //cout << "Sum of weights(Should be 1):" << endl;
263 //cout << "Sum of weights(Should be 1):" << endl;
264 //cout << "Sum of weights(Should be 1):" << endl;
265 //cout << "Sum of weights(Should be 1):" << endl;
266 //cout << "Sum of weights(Should be 1):" << endl;
267 //cout << "Sum of weights(Should be 1):" << endl;
268 //cout << "Sum of weights(Should be 1):" << endl;
269 //cout << "Sum of weights(Should be 1):" << endl;
270 //cout << "Sum of weights(Should be 1):" << endl;
271 //cout << "Sum of weights(Should be 1):" << endl;
272 //cout << "Sum of weights(Should be 1):" << endl;
273 //cout << "Sum of weights(Should be 1):" << endl;
274 //cout << "Sum of weights(Should be 1):" << endl;
275 //cout << "Sum of weights(Should be 1):" << endl;
276 //cout << "Sum of weights(Should be 1):" << endl;
277 //cout << "Sum of weights(Should be 1):" << endl;
278 //cout << "Sum of weights(Should be 1):" << endl;
279 //cout << "Sum of weights(Should be 1):" << endl;
280 //cout << "Sum of weights(Should be 1):" << endl;
281 //cout << "Sum of weights(Should be 1):" << endl;
282 //cout << "Sum of weights(Should be 1):" << endl;
283 //cout << "Sum of weights(Should be 1):" << endl;
284 //cout << "Sum of weights(Should be 1):" << endl;
285 //cout << "Sum of weights(Should be 1):" << endl;
286 //cout <<
```

Case 4 (Simulation run for 100,000 scenarios):

```

100 // we have set stack limit to 100MB (100MB) to avoid overflow
101 Matrix<double> m(1000000, 1000000, 0);
102 for (int i = 0; i < 100000; i++)
103     Matrix<double> PseudoInvert();
104
105 // Use C++ seed < endl>;
106 Random101 m(10000, 64, (uint) seed);
107 Matrix<double> m1, m2; Samples <- gen1.generate(mrg, 50);
108 // cout << "Data are the covariance matrix << endl>;
109 // cout << Samples << endl>;
110
111 // Calculate covariance matrix of each scenario
112 Matrix<double> m1Samples = calculateMatrix<double>(Samples);
113 Matrix<double> m2Samples = calculateCovarianceMatrix<double>(Samples);
114 Matrix<double> m1InvSamples = calculateInverseCovarianceMatrix<double>(Samples);
115
116
117 OUTPUT:
118
119
120
121
122
123
124
125
126
127
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1
```

The screenshot provided below indicates eigen library has a 120kb threshold on creating a matrix on a stack, so first line nulls the limit and the second line of code expands the stack threshold of windows C++ compiler, VSCode specifically.

```
project.cpp > main()
1 #define EIGEN_STACK_ALLOCATION_LIMIT 0 // No stack limit
2 #pragma comment(linker, "/STACK:209715200,209715200") // To set reserve and commit sizes
~
```

For the 10, 1000, 10,000, and 100,000 scenarios, we have presented four examples. In order to achieve the target mean of 0.02, which is equivalent to a weekly return of 2 percent, we have computed the ideal weights for each scenario, as well as the covariance matrix and the inverse covariance matrix. Finally, the optimal weights for the portfolio are extracted to a CSV file.

In conclusion, the optimum weights are calculated by taking the average of the optimum weights obtained in each individual instance. It has come to our attention that the optimal weights tend to converge around a particular amount as the number of situations increases. This demonstrates how Monte Carlo simulation may be used to achieve optimal weights for the goal mean that has been set.

V. FUTURE WORK

The project can be further modified to carry out the calculations to find Value at Risk (VaR) and Average Value at Risk (AVaR) in portfolio optimization allowing a more comprehensive evaluation of risk thus enabling investors to balance risk and return as these methods provides an additional layer of risk assessment offering insights into the average loss magnitude over time. This method can be further compared with the more sophisticated optimization techniques beyond mean-variance analysis, such as Black-Litterman model, Bayesian optimization, or machine learning-driven portfolio allocation strategies to enhance portfolio performance.

VI. REFERENCES

1. Cong, F., and C. Oosterlee. "Multi-period mean–variance portfolio optimization based on Monte-Carlo simulation". 2016.
<https://ir.cwi.nl/pub/24376/24376D.pdf>
2. He, Yifan, and C. Aranha. "Solving Portfolio Optimization Problems Using MOEA/D and Levy Flight". 2020. <https://arxiv.org/pdf/2003.06737>
3. K. Postek and D. Hertog, "Computationally Tractable Counterparts of Distributionally Robust Constraints on Risk Measures", 2014. Available:
https://research.tilburguniversity.edu/files/3035948/2014_031.pdf
4. Eeva Vilkkumaa, Juusi Liesio, Ahti Salo, Leena Ilmola, "Scenario-based portfolio model for building robust and proactive strategies", 2016
5. Gianfranco Guastaroba, Renata Mansini, M Grazia, "On the effectiveness of scenario generation techniques in single-period portfolio optimization", 2007
6. Sovan Mitra, Sungmook Lim, Andreas Karathanasopoulos, "Regression based scenario generation: Applications for performance Management", 2014
7. Pier Francesco Procacci and Tomaso Aste, "Portfolio Optimization with sparse multivariate modelling", Depart of Computer Scienc, UCL, London, UK, 2019
8. Louis K. Chan, Jason. K and Josef. L, "On Portfolio Optimization: Forecasting Covariances and Choosing the Risk Model", University of Illinois at Urbana-Champaign, 2021
9. Jonathan Huan Jing and Sie Long Kek, "A simulation optimization model for portfolio selection problem with quadratic programming technique", Unisersit Tun Hussein, 2020